

FlaPLeT: A Full-Stack Web Platform for End-to-End Time Series Data Processing and Machine Learning in Solar Flare Prediction

MohammadReza EskandariNasab^{a,*}, Shah Muhammad Hamdi^a, Soukaina Filali Boubrahimi^a

^a*School of Computing, Utah State University, Logan, UT 84322, USA.
Emails: {reza.eskandarinasab, s.hamdi, soukaina.boubrahimi}@usu.edu*

Abstract

Solar flare prediction is a central challenge in space weather forecasting with critical implications for satellite operations and power grid stability. FlaPLeT is an open-source, full-stack web platform that enables end-to-end machine learning workflows for multivariate time series analysis without requiring coding expertise. Built with React, Django, Celery, and PostgreSQL, the system unifies dataset preprocessing, data augmentation, functional network (graph) generation, and machine learning model training into asynchronous tasks with downloadable datasets, models, and JSON reports. Deployment on a dedicated Windows server integrates NGINX, Waitress, Redis, TLS encryption, and reCAPTCHA for secure and scalable performance. FlaPLeT lowers barriers for heliophysicists to apply machine learning models to photospheric magnetic field data, enabling systematic studies of how preprocessing strategies and hyperparameter choices affect the accuracy of solar flare prediction. Beyond heliophysics, its modular architecture provides a reusable blueprint for AI-driven services in domains such as biomedical signal processing and finance, as well as modern applications, including large language model inference, diffusion-based media generation, and agentic AI workflows.

Keywords: Full-stack web platform, Asynchronous task execution, Time series analysis, AI services, Space weather prediction

*Corresponding author. ORCID: 0009-0004-0697-3716

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.0 (latest commit on main branch)
C2	Permanent link to code/repository used for this code version	https://github.com/samresume/FlaPLeT
C3	Permanent link to Reproducible Capsule	Not available
C4	Legal Code License	MIT License
C5	Code versioning system used	Git
C6	Software code languages, tools, and services used	Python: Django, Celery, Django REST Framework, scikit-learn, TensorFlow, NumPy, Pandas, SciPy, imbalanced-learn, Node2Vec; JavaScript/Node.js: React, MUI; Databases and services: PostgreSQL, Redis, Waitress, Nginx
C7	Compilation requirements, operating environments & dependencies	Requires Python 3.8+ (with pip) and Node.js (with npm); Backend dependencies listed in <code>backend/requirements.txt</code> ; Frontend dependencies listed in <code>frontend/package.json</code> ; Requires PostgreSQL, Redis, Waitress, and NGINX for deployment; Tested on a Windows server with static-IP port forwarding.
C8	If available, link to developer documentation/manual	Documentation: https://github.com/samresume/FlaPLeT#readme ; Video (Architecture overview): https://youtu.be/g-JB8GZnXdI&t=46s ; Video (Functionality demo): https://youtu.be/gbGt0xnmhVw
C9	Support email for questions	sameskandarinasab@gmail.com

Table 1: Code metadata (mandatory)

1. Motivation and significance

Space weather forecasting, particularly solar flare prediction, is a central heliophysics challenge. Solar flares are sudden bursts of electromagnetic radiation and energetic particles produced by magnetic reconnection and of-

ten linked to coronal mass ejections; they can disrupt radio communication, GPS, and satellites, increase radiation exposure for astronauts and high-altitude flights, and trigger geomagnetically induced currents that threaten power grids on Earth [?]. Traditional systems such as NOAA’s FlareCast rely on hand-crafted features and threshold rules from sunspot classifications and magnetic snapshots. In contrast, multivariate time series (MVTs) of photospheric magnetic parameters [? ?] derived from SDO/HMI vector magnetograms [? ?] track active-region evolution via measures of flux, helicity, and shear. The SWAN-SF dataset [?] standardizes these sequences and links them to GOES flare labels [?], enabling ML studies. Consequently, ML models, especially recurrent and graph-based networks, learn temporal dependencies beyond hand-engineered thresholds, improving predictive skill [? ?]. However, applying these methods requires Python programming, deep learning expertise, and GPU access, which pose barriers for many heliophysicists.

FlaPLeT addresses these challenges by delivering a full-stack, no-code web platform that unifies preprocessing, data augmentation, functional network construction, and ML model training into modular workflows. Tasks run asynchronously, and each produces JSON reports and downloadable outputs. This design allows researchers to focus on solar flare physics rather than coding or infrastructure. FlaPLeT contributes to scientific discovery not only in solar flare forecasting but also as a model for how other disciplines can overcome barriers between domain expertise and machine learning. Many fields generate high-dimensional or sequential data, but their adoption of AI is limited by insufficient ML programming expertise and hardware resources. In biomedical signal processing [?], EEG or ECG data could be preprocessed and modeled through a similar no-code system to support disease detection. In climate science [?], large-scale atmospheric and oceanic datasets could be analyzed with comparable pipelines to improve extreme weather prediction. Beyond research, FlaPLeT’s modular architecture is applicable to a wide range of online AI services that rely on asynchronous task execution. Chatbot systems such as ChatGPT [?] and Gemini [?] run large models for inference, while image and video generation platforms [?] and agentic AI systems, which depend on multi-step reasoning workflows, can similarly benefit from such designs. FlaPLeT therefore provides a reusable blueprint for scientific software and everyday AI applications that may serve millions of users.

As an example of the experimental setting, FlaPLeT was made available through the public portal at <https://flaplet.org> for a limited time. During this initial deployment, users created accounts, logged into their personalized dashboards, and accessed four core tasks: (i) dataset upload and

preprocessing, (ii) data augmentation, (iii) functional network (graph data) generation, and (iv) model training. Each task executed asynchronously in the backend, with status, metadata, and progress displayed on the dashboard. Users could retain or delete completed tasks, and for every task both a structured report (in JSON) and the resulting products, such as processed datasets, graph files, or trained models, were stored and made available for download.

FlaPLeT is built on established technologies that ensure scalability and maintainability. Django [?], a widely used Python web framework, has been successfully applied in scientific web applications due to its modular design and emphasis on security and reproducibility. Celery [?], implemented in Python, provides distributed asynchronous task execution and integrates naturally with Django. Since most modern AI algorithms and frameworks are implemented in Python, combining Django, Celery, and Python-based AI libraries provides a natural environment for integrating advanced and long-running machine learning pipelines directly into the platform. React [?] powers the user interface, enabling responsive dashboards for complex data-driven workflows. PostgreSQL [?] offers robust ACID-compliant storage, and Redis [?] accelerates task management through high-performance in-memory caching. Nginx [?] ensures high-throughput and secure web serving. Together, these technologies provide a reliable foundation for AI-driven scientific platforms, making FlaPLeT both innovative in algorithmic design and rigorous in engineering practice, with the robustness and scalability needed for future extensions.

2. Software description

Based on the system architecture illustrated in Figure 1, the web platform *FlaPLeT* (**Flare Prediction by Learning from Time Series**) is implemented as a modular web application. The architecture integrates a React-based frontend, a Django backend, asynchronous task processing with Celery, and a PostgreSQL relational database. Each technology was chosen for its popularity, maturity, and ability to support scalable and maintainable AI-driven systems. The complete source code of the platform, including the frontend, backend, Celery task pipelines, and integrated AI models, is publicly available on GitHub¹. For deployment, the repository also provides configuration files for NGINX.

¹<https://github.com/samresume/FlaPLeT>

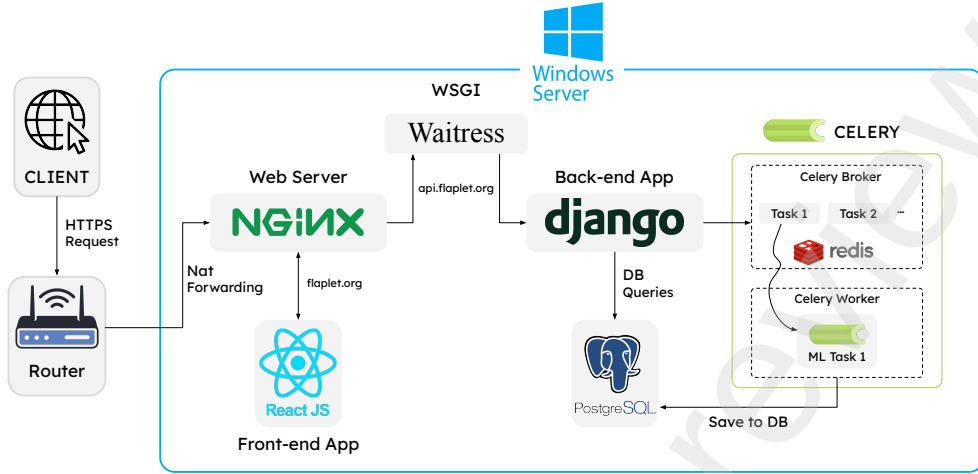


Figure 1: System architecture of FlaPLeT. User requests are first processed by NGINX, which serves the React frontend from <https://flaplet.org> and routes API traffic to the Django backend hosted at <https://api.flaplet.org>. The backend, running on Waitress for Windows, communicates with PostgreSQL for data persistence and dispatches long-running tasks to Celery workers coordinated through Redis. Processed datasets, trained models, and reports are stored as downloadable artifacts, ensuring scalability and reproducibility.

2.1. Software architecture

The frontend of FlaPLeT is built using **React JS**, a widely adopted JavaScript library known for its component-based architecture, declarative rendering, and efficient virtual DOM updates. These features enable the creation of interactive and responsive dashboards that scale cleanly as new functionalities are added. Styling and layout are implemented with Google’s Material UI (MUI) [?], which provides pre-designed, accessible components for a consistent user experience. The frontend was deployed under the public domain <https://flaplet.org>.

User requests are initially handled by **NGINX**, a high-performance web server and reverse proxy that routes HTTPS traffic, serves static frontend files, and forwards API calls to the backend. Its event-driven, non-blocking design makes it highly efficient under concurrent load. However, because Django is not designed to communicate directly with web servers such as NGINX, a dedicated WSGI server is required. The Web Server Gateway Interface (WSGI) standard defines how Python applications exchange requests and responses with web servers. In this deployment, the backend application is hosted with **Waitress** [?], a production-grade WSGI server optimized for Windows environments. Waitress provides multithreaded request han-

dling and integrates seamlessly with Django, making it a practical choice for ensuring stability and performance.

The backend is implemented using the **Django** web framework, known for its “batteries-included” philosophy that provides built-in tools for authentication, ORM-based database access, form handling, and security features. Django works seamlessly with SQL-based databases through its Object Relational Mapper (ORM), which automatically translates Python objects into SQL queries and manages schema migrations. This abstraction allows developers to interact with the database using high-level Python code while still benefiting from the robustness and performance of relational databases such as PostgreSQL. For communication with the frontend, Django exposes **RESTful APIs** [?] through the Django REST Framework (DRF). REST (Representational State Transfer) is a widely adopted standard for designing web services that enables different systems to exchange data using simple HTTP methods such as GET, POST, PUT, and DELETE. DRF simplifies this process by automatically mapping database models to API endpoints, handling serialization and deserialization of JSON objects, and enforcing authentication and permissions. This design allows the frontend and backend to communicate efficiently while maintaining scalability. The backend services, including all REST API endpoints, were deployed under the domain <https://api.flaplet.org>.

Web requests in Django, served through a WSGI server such as Waitress, are optimized for short, synchronous operations such as rendering pages or handling small data transactions. Executing compute-intensive jobs such as preprocessing or ML model training directly within this cycle could freeze the backend until the task completes. Moreover, these scientific workloads often require access to specialized resources (e.g., GPUs) and benefit from queuing and prioritization, which standard web worker pools are not designed to support. To overcome these limitations, long-running operations are delegated to **Celery**, a distributed task queue system for managing asynchronous and parallel workloads [?]. Celery executes tasks through worker processes that may operate concurrently across multiple CPU cores, while **Redis** functions as the message broker responsible for queuing, scheduling, and tracking execution states. This separation allows computationally intensive tasks to run independently of the web request cycle, keeping the frontend responsive and ensuring that users can continue interacting with the platform. Since Celery is implemented in Python, it integrates naturally with Django and facilitates the direct incorporation of machine learning frameworks such as TensorFlow [?], PyTorch [?], and scikit-learn [?]. Its support for horizontal scaling and fault tolerance further establishes Celery as a robust foundation for embedding advanced AI pipelines within the platform.

The platform relies on **PostgreSQL**, a robust open-source relational database widely adopted in both scientific and enterprise applications. In FlaPLeT, the relational schema (Figure 2) organizes core entities such as users, datasets, and machine learning tasks. It also includes supporting entities such as files, bug reports, feedback, and admin messages to improve the user experience, while foreign keys enforce consistency across the workflow. To avoid performance bottlenecks, large artifacts such as datasets, reports, and trained models are stored on disk and referenced within the database. The efficiency of this schema design is central to the platform, as well-structured tables and relationships preserve data integrity and ensure that queries remain fast and scalable as the number of users and tasks grows.

Together, these components form a cohesive and scalable architecture that enables researchers to execute end-to-end AI workflows through a secure, maintainable, and web-accessible environment.

2.1.1. Security and deployment features

FlaPLeT was initially designed for deployment on a dedicated server rather than relying on a cloud provider, in order to maintain full control over configuration, networking, and security. The system runs on a dedicated Windows server with a static IPv4 address, enabling persistent domain mapping and reliable access. Public DNS records map `flaplet.org` (frontend) and `api.flaplet.org` (backend) to this static IP, while router-level NAT forwarding directs incoming traffic on ports 80 and 443 to the internal host. NGINX is configured to terminate TLS using ZeroSSL certificates and to enforce HTTPS redirection. By using TLS (commonly referred to as SSL in certificate terminology), all communication between clients and the platform is encrypted and protected against interception. Multiple layers of security are implemented within the application stack. Django provides built-in authentication with strong password policies, session management, and CSRF protection. Google reCAPTCHA v2 is integrated into login and submission forms to protect against brute-force attacks and automated abuse. PostgreSQL and Redis are firewalled and only accessible internally, ensuring that neither the database nor the task queue is directly exposed to the internet.

2.2. Software functionalities

FlaPLeT provides an end-to-end environment for working with MVTs data, specifically designed for solar flare prediction. The platform is organized into four main task categories: dataset upload and preprocessing, data augmentation, functional network (FN) generation, and machine learning classification (model training). All tasks are executed asynchronously using Celery

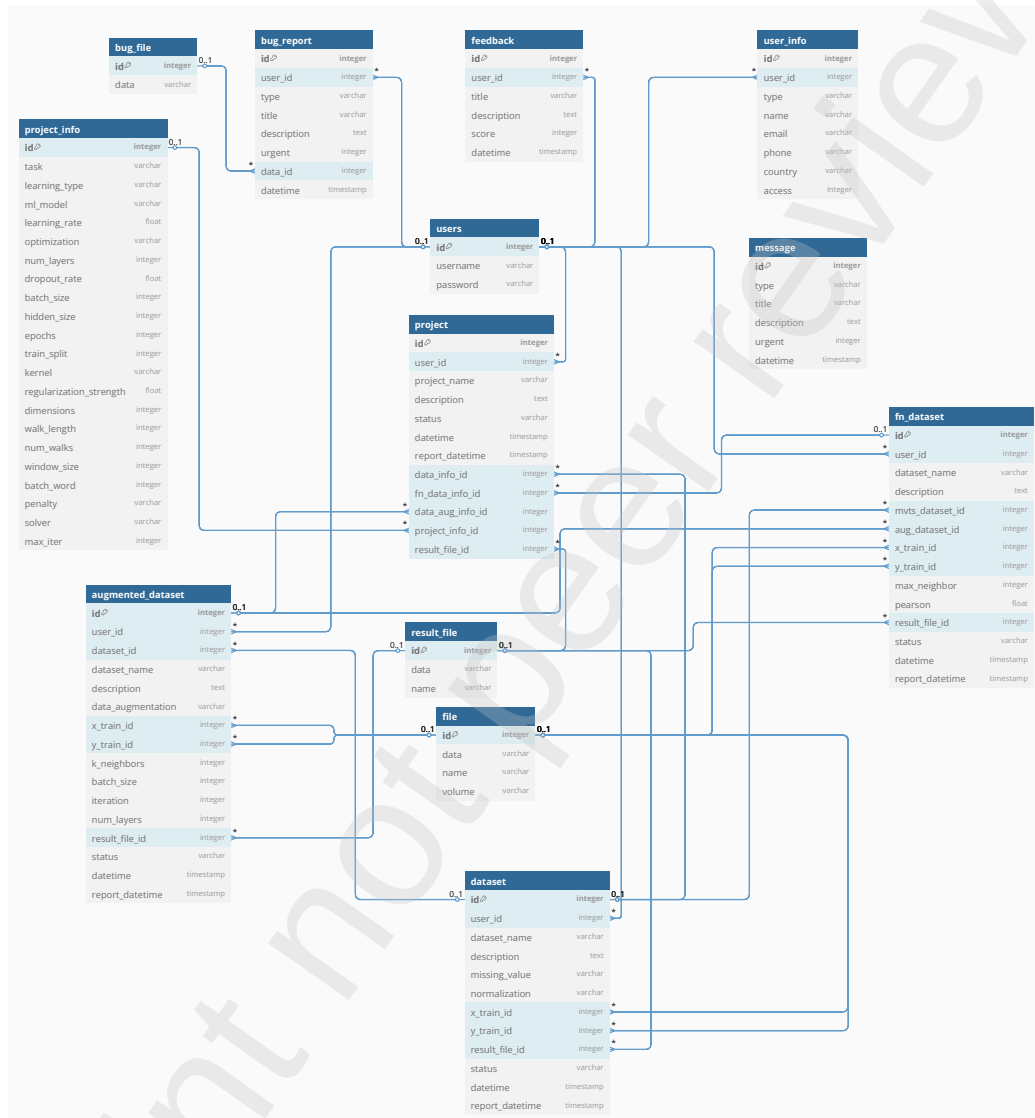


Figure 2: Relational database schema of FlaPLeT. The design captures users (users and user_info), datasets, augmented datasets, functional network datasets (fn_dataset), machine learning tasks (project and project_info), and reports (result_file), along with files and user experience entities such as bug reports, feedback, and admin messages. Relationships are enforced through foreign keys, enabling scalability, version control, and efficient management of scientific workflows.

workers, ensuring that users can continue interacting with the system while computations run in the background.

Dataset Upload and Preprocessing. Users upload MVTS datasets in .pkl format, where missing values can be handled by mean imputation or removal. Normalization options include z-score, min-max scaling, or none. Each preprocessing task generates a cleaned dataset and a summary report detailing data shape, label distribution, and preprocessing statistics, both of which are available for download.

Data Augmentation. For this task, users first select one of their previously uploaded datasets. The platform supports two augmentation methods that balance dataset classes by generating synthetic minority samples: SMOTE [?], with the number of neighbors as a configurable parameter, and TimeGAN [?], which uses a generative adversarial framework with configurable batch size, number of layers, and training iterations. Each augmentation task produces a new balanced dataset along with a summary report detailing dataset shape and class distributions before and after augmentation.

Functional Network Generation. For this task, users can select either a previously uploaded dataset or a balanced dataset generated through augmentation. The module converts MVTS data into graph representations by computing pairwise Pearson correlations between features. Users specify a correlation threshold and a maximum number of neighbors per node. Each task outputs a weighted undirected graph in .gpickle format and a summary report detailing node count, edge count, and label distribution.

Machine Learning Classification. For this task, users select either a previously uploaded dataset, a balanced dataset generated through augmentation, or a graph dataset created via functional network generation. Depending on the dataset type, three model families are supported. For MVTS data, the platform provides a GRU model [?] with configurable parameters such as number of layers, dropout rate, learning rate, and number of epochs. As an alternative, users may choose an SVM [?], with configurable kernel type and regularization strength. For graph-based datasets, the system applies Node2Vec embeddings [?] combined with logistic regression, with hyperparameters including embedding dimensions, number of walks, regularization penalty, solver, and maximum iterations. In all cases, users also define the train-test split ratio. Each classification task outputs a trained model and a comprehensive report containing evaluation metrics such as accuracy, precision, recall, AUC, TSS, HSS, and GS.

2.3. Sample code snippets analysis

Figure 3 illustrates the integration of Django, Celery, and AI logic within the FlaPLeT platform. The workflow begins in the Django layer (Figure 3a),

views.py

```

1 2 class DatasetViewSet(viewsets.ViewSet):
3 3     def create(self, request):
4 4         name = request.data.get("name")
5 5         missing_value = request.data.get("missing_value")
6 6         normalization = request.data.get("normalization")
7 7         x_train = request.FILES.get("x_train")
8 8         y_train = request.FILES.get("y_train")
9 9         if not all([name, missing_value, normalization,
10 10                    x_train, y_train]):
11 11             return Response(
12 12                 {"error": "All fields must be provided."},
13 13                 status=400
14 14             )
15 15         dataset = Dataset.objects.create(
16 16             name=name,
17 17             user=request.user,
18 18             missing_value=missing_value,
19 19             normalization=normalization,
20 20             x_train=x_train,
21 21             y_train=y_train,
22 22             status="queued",
23 23         )
24 24         from .celeryTask import celeryTask
25 25         celeryTask.delay(id=str(dataset.id), type="pp")
26 26         return Response(
27 27             {
28 28                 "message": "Dataset uploaded and preprocessing started.",
29 29                 "id": str(dataset.id)
30 30             },
31 31             status=201
32 32         )

```

```

# celery.py
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE",
    "backend.settings")

app = Celery("backend")
app.config_from_object("django.conf:settings",
    namespace="CELERY")
app.autodiscover_tasks()

# celeryTask.py

@shared_task(bind=True, name="Flaplet.celeryTask")
def celeryTask(self, *, id: str, type: str):
    try:
        if type == "pp":
            from .pp import process_dataset
            process_dataset(id=id)
        else:
            raise ValueError(f"Unknown task type: {type}")
    except Exception as e:
        raise

```

(a) Django view for dataset upload and preprocessing. (b) Celery configuration and task dispatcher.

pp.py

```

1  def process_dataset(*, id: str):
2  ds = Dataset.objects.get(id=id)
3
4  with ds.x_train.open("rb") as fx, ds.y_train.open("rb") as fy:
5      X = pickle.load(fx)
6      y = pickle.load(fy)
7
8  if ds.missing_value == "mean_impute":
9      # mean impute NaNs
10     if X.ndim == 3:
11         m = np.nanmean(X, axis=(0, 1), keepdims=True)
12         X = np.where(np.isnan(X), m, X)
13     else:
14         m = np.nanmean(X, axis=0, keepdims=True)
15         X = np.where(np.isnan(X), m, X)
16     if ds.normalization == "zscore":
17         if X.ndim == 3:
18             mu = X.mean(axis=(0, 1), keepdims=True)
19             sd = X.std(axis=(0, 1), keepdims=True) + 1e-8
20         else:
21             mu = X.mean(axis=0, keepdims=True)
22             sd = X.std(axis=0, keepdims=True) + 1e-8
23         X = (X - mu) / sd
24
25     out_dir = Path(settings.MEDIA_ROOT) / "datasets" / str(ds.id)
26     out_dir.mkdir(parents=True, exist_ok=True)
27
28     with (out_dir / "X_processed.pkl").open("wb") as fxp:
29         pickle.dump(X, fxp)
30     with (out_dir / "y_processed.pkl").open("wb") as fyp:
31         pickle.dump(y, fyp)
32
33     ds.status = "completed"
34     ds.save(update_fields=["status"])

```

(c) AI preprocessing function executed by Celery.

Figure 3: Integration of Django, Celery, and AI logic: (a) Django view receives a POST request from the React frontend for a dataset upload and enqueues a preprocessing task, (b) Celery manages task distribution and dispatching, and (c) the AI preprocessing code runs asynchronously, completing the computation, saving the processed files, and updating the dataset status.

where the `DatasetViewSet` receives an HTTP POST request containing metadata and dataset files. After validating the inputs, Django records the dataset object in PostgreSQL and delegates the preprocessing work to the task queue by invoking `celeryTask.delay`.

Celery, configured as a distributed task queue (Figure 3b), decouples long-running computations from the web request cycle. When a preprocessing job is enqueued, the broker (Redis) assigns it to available Celery workers. This ensures that the web server remains responsive while queued tasks can scale across CPU cores or even multiple servers.

The AI-specific preprocessing logic (Figure 3c) is then executed by Celery workers. In this case, the `process_dataset` function loads the uploaded data, applies mean imputation for missing values, performs normalization (e.g., z-score), and writes the processed datasets back to storage. Once completed, the dataset status is updated in the database, and the results, including both processed data and reports, are made available through the web interface.

3. Illustrative Examples

To demonstrate the functionality of *FlaPLeT*, we provide a series of examples illustrating the main components of the platform. Figure 4 shows the primary user-facing workflow. Users begin at the landing and login pages, where authentication is secured by Google reCAPTCHA. After logging in, the dashboard provides a centralized view of all submitted tasks, categorized by type and status. Dedicated forms allow the creation of tasks such as data augmentation or ML model training. Tasks can be viewed on dedicated pages, and task cards update asynchronously as jobs progress, ensuring a responsive user experience.

Furthermore, the Django framework provides an **administrative interface** that is automatically generated and customizable. This Django Admin panel (Figure 5) enables administrators to manage users, datasets, task records, and metadata through a secure web interface. Its availability out-of-the-box reduces development overhead and ensures that administrators can efficiently monitor, debug, and control system behavior without requiring direct database access.

Beyond the frontend interactions, each task produces structured JSON reports that capture input parameters, data transformations, and results. Figure 6 highlights three examples. These reports not only provide transparency and enable export to external pipelines, but also help heliophysics researchers assess how changes in hyperparameters affect the accuracy of solar flare pre-

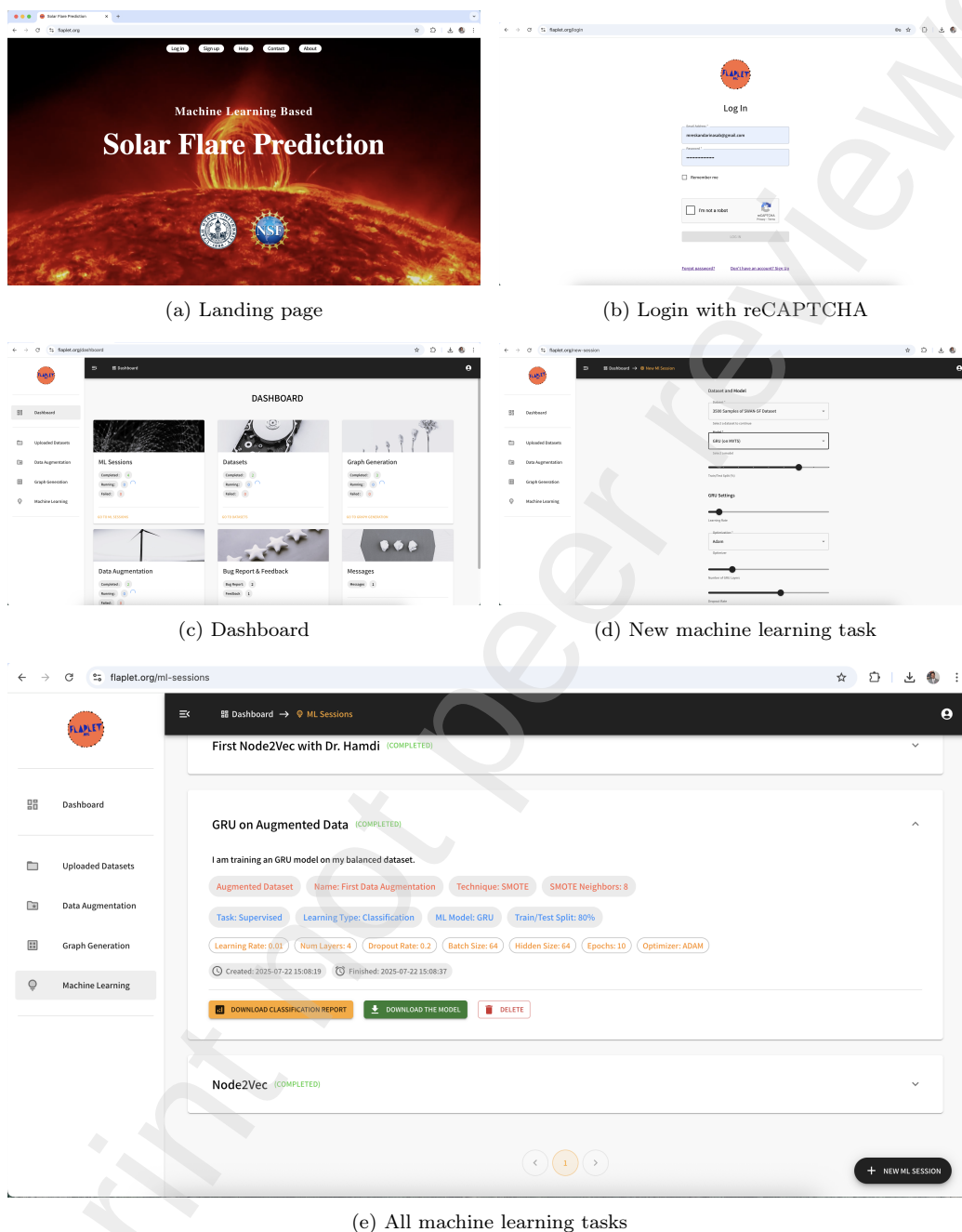
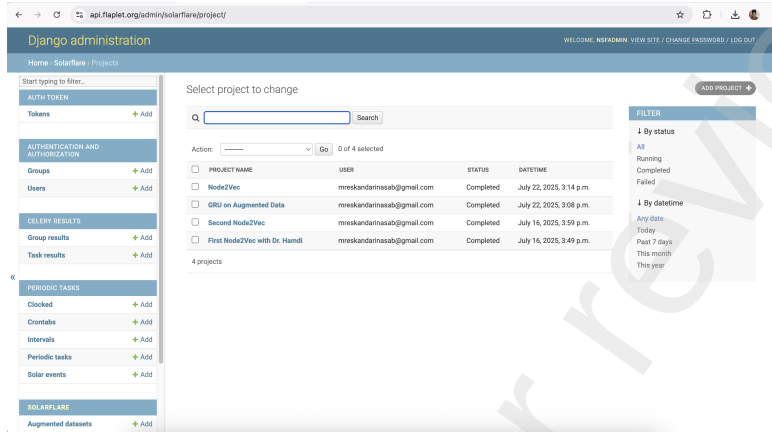
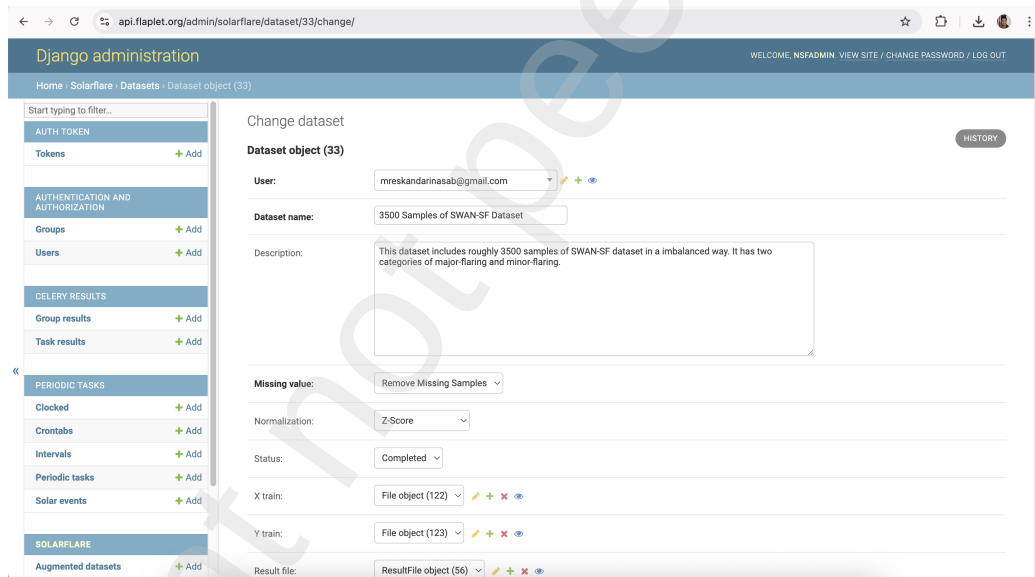


Figure 4: Overview of the FlaPLeT web platform. Main screens include the landing page (with access to login, signup, help, contact, and about pages), login with reCAPTCHA, the dashboard displaying the overall status of all tasks, the new machine learning task page where users select a preprocessed, augmented, or graph dataset and configure model type and hyperparameters, and the task management page showing metadata for each ML job with options to download reports or trained models.



(a) Machine learning task records



(b) Dataset record editing view

Figure 5: Django Admin interface of FlaPLeT, designed to provide detailed information about each table and its data with options for search and filtering. It offers administrative views for site management, including the ability to modify table records, manage user accounts (activation/deactivation), review bug reports and feedback, and broadcast messages to all users.

diction and serve as a training tool that deepens their understanding of machine learning workflows.

To further illustrate the platform, we provide two supplementary videos² that walk through the system design and demonstrate end-to-end workflows.



Figure 6: Examples of JSON reports automatically generated by FlaPLeT: (a) GRU model training on an 80/20 train-test split, including confusion-matrix counts and evaluation metrics; (b) preprocessing with Z-normalization and missing-value removal, reporting data shape, class balance, and highly correlated features; and (c) functional-network construction from MVTs data, summarizing the number of nodes, edges, and label distribution.

4. Impact

By providing advanced machine learning pipelines through a no-code interface, FlaPLeT enables systematic studies that would otherwise require extensive custom coding. Researchers can now evaluate how preprocessing choices and class-balancing strategies influence forecast skill (e.g., TSS and

²Platform Architecture Overview: <https://youtu.be/g-JB8GZnXdI&t=46s>; Platform Functionality Demo: <https://youtu.be/gbGt0xnmhVw>

HSS) across different flare datasets, compare sequential deep learning models on multivariate time series with graph-based approaches derived from functional networks [?], assess robustness under domain shifts across solar cycles and active regions [?], and conduct controlled ablation studies or hyperparameter sensitivity analyses using standardized reports and reproducible data splits. Daily research practice is also affected. Routine work for heliophysicists shifts from developing and maintaining scripts to selecting preprocessing, augmentation, and classification options through a web interface. This substantially reduces the time from experimental design to results, with many tasks completing within minutes. The products of these tasks, such as processed datasets and trained ML models, can be further downloaded and used for real-time solar flare prediction.

FlaPLeT was made publicly available at <https://flaplet.org> for a limited period, where internal collaborators conducted complete experiments. During this phase, access was primarily provided to the Utah State University (USU) School of Computing, including PhD students and faculty working on space weather prediction. Wider availability for researchers in other institutions and regions is planned as the system evolves, with new versions expected to incorporate additional algorithms and more comprehensive report generation. The full source code, including deployment assets, is openly available ensuring that the system can be self-hosted, extended, or reproduced by other groups. This openness lays a foundation for developers to create variants for other research domains, adapt the platform for Linux-server environments, incorporate additional machine learning and data-processing algorithms, or enhance its security features. Because the architecture is domain-agnostic, its applications extend beyond heliophysics to areas such as biomedical signal analysis, climate reanalysis, and finance [?], where similar challenges in preprocessing, modeling, and transparent reporting are prevalent. Although developed for research, the architecture mirrors patterns used in production AI services: a web frontend, authenticated APIs, asynchronous workers for long-running jobs, and a relational store for metadata and outputs. The same design supports chatbot inference endpoints [?], diffusion-based image or video generation, and agentic multi-step workflows that require queued execution and result retrieval. Organizations could adapt FlaPLeT as a blueprint for internal analytics portals or domain-specific AI services.

5. Conclusions

FlaPLeT delivers a secure, user-friendly, and accessible web platform that integrates all stages of the machine learning pipeline for space weather prediction, including preprocessing, data augmentation, functional network gen-

eration, and machine learning model training. Its no-code, interactive interface allows researchers to design and execute customized workflows without requiring programming expertise, reducing the entry barrier for astrophysicists and practitioners. The modular architecture supports scalability, reproducibility, and adaptability, while asynchronous execution with Celery ensures responsiveness during computationally demanding tasks. Beyond solar flare forecasting, FlaPLeT establishes a flexible foundation that can be extended to a wide range of emerging AI services, including large language model inference, multimodal agents, generative media systems, real-time translation, scientific simulations, and other cloud-based AI applications developed by major research and industry leaders. By releasing the full source code as open source, FlaPLeT fosters transparency, collaboration, and reusability, offering a sustainable blueprint for future scientific web applications that integrate AI-driven workflows into accessible online environments.

CRedit authorship contribution statement

MohammadReza EskandariNasab: Conceptualization, Methodology, Software, Validation, Data curation, Writing – original draft, Writing – review & editing, Visualization, Project administration. **Shah Muhammad Hamdi:** Conceptualization, Methodology, Validation, Resources, Writing – review & editing, Supervision, Project administration, Funding acquisition. **Soukaina Filali Boubrahimi:** Conceptualization, Resources, Writing – review & editing, Supervision, Funding acquisition.

Acknowledgements

FlaPLeT is primarily supported by the National Science Foundation (NSF) Office of Advanced Cyberinfrastructure (OAC) under award #2305781 (PI: Shah Muhammad Hamdi). Soukaina Filali Boubrahimi acknowledges support from the NSF Geosciences (GEO) Directorate, Division of Atmospheric and Geospace Sciences (AGS), under awards #2204363, #2240022, and from the NSF RISE program, Integrative and Collaborative Education and Research (ICER), under award #2530946. Shah Muhammad Hamdi acknowledges additional support from the NSF Geosciences (GEO) Directorate, Division of Atmospheric and Geospace Sciences (AGS), under award #2301397, and from the NSF RISE program, ICER, under award #2530946.

Declaration of competing interest

The authors declare that they have no known competing financial interests that could have appeared to influence the work reported in this paper.

Supplementary material

Supplementary material associated with this article includes a 40-minute video demonstration. The video explains the architecture of the FlaPLeT platform and tests all implemented functionalities. This combined video is also available as two separate YouTube videos, with links provided in Section 3.